

RISC-V MANUAL

RISC-V ISA SIMULATOR

Angstromers Engineering Solutions Pvt Ltd

LIST OF LAB EXPERIMENTS

No.	Projects	Page No.
1	C program using RISC-V ISA Simulator for - Addition - Subtraction - Multiplication - Division - Write C Code - Simulate the generated ELF file using RISC-V SPIKE simulator	4 7 9 11
2	C program for DFT using RISC-V ISA Simulator - Write C Code - Simulate the generated ELF file using RISC-V SPIKE simulator	15
3	C program for FFT using RISC-V ISA Simulator - Write C Code - Simulate the generated ELF file using RISC-V SPIKE simulator	18
4	C program Convolution using RISC-V ISA Simulator - Write C Code - Simulate the generated ELF file using RISC-V SPIKE simulator	21
5	C program Cholesky Decomposition using RISC-V ISA Simulator Write C Code Simulate the generated ELF file using RISC-V SPIKE simulator	23
6	C program for bubble sort using RISC-V ISA Simulator - Write C Code - Simulate the generated ELF file using RISC-V SPIKE simulator	25
7	C program for binary search using RISC-V ISA Simulator - Write C Code - Simulate the generated ELF file using RISC-V SPIKE simulator	27

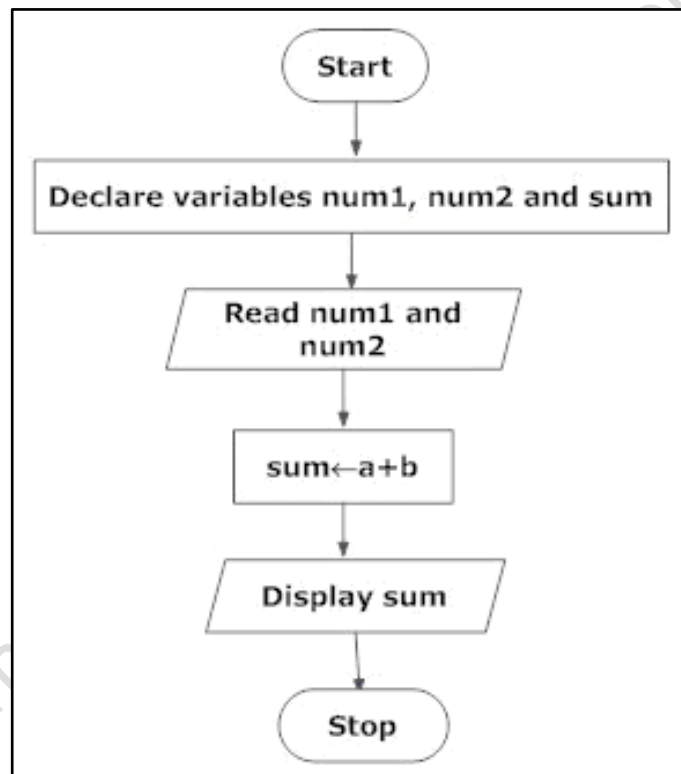
1: C program to implement addition operations using RISC-V Simulator

AIM: To write a C program for addition and run simulation using RISC-V Simulator.

Tool Required:

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Flow chart for Addition



M1. Creating a Work space :

- To create a new folder in the directory type the command mkdir followed by the folder name, in which the saved program will be stored

```
copi-001@copi001-OptiPlex-7050:~$ mkdir Addition
```

- To access the newly created folder type the command cd followed by the folder name

```
copi-001@copi001-OptiPlex-7050:~$ cd Addition
```

M2. Creating Source Codes

- To create a new file to write the c code using editor type command nano followed by .c extension

```
copi-001@copi001-OptiPlex-7050:~/addition$ nano add.c
```

- A Blank Document opens up into which the following source code can be typed down.

C code for Addition

```
#include <stdio.h>
int main() {
```

```
    int number1, number2, sum;
    printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2);
    sum = number1 + number2; // calculate the sum
    printf("%d + %d = %d", number1, number2, sum);
    return 0;
```

```
}
```

- Click on Ctrl+S to save the code
- Click on Ctrl+X to exit from text editor

M3. Functional Simulation:

- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file

```
tiPlex-7050:~/spike/Embedded/add$ riscv64-unknown-elf-gcc add.c
```

- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/add$ spike pk a.out
bbl loader
Enter two integers: 2 5
2 + 5 = 7copi-001@copi001-OptiPlex-7050:~/spike/Embedded/add$
```

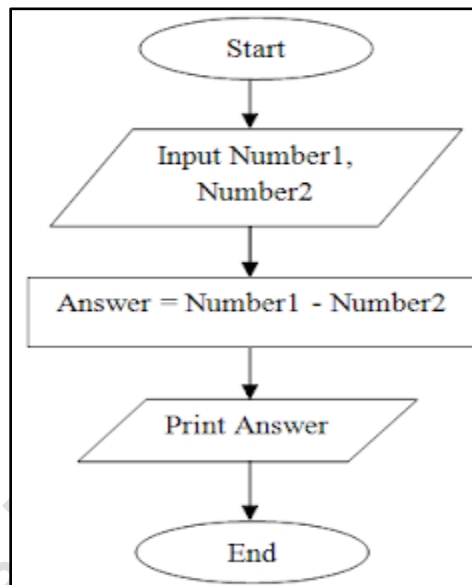
1b : C program to implement subtraction operations using RISC-V Simulator

AIM: To write a C program for subtraction and run simulation using RISC-V Simulator.

Tool Required :

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Flow chart for Subtraction



Creating a Work space :

- To create a new folder in the directory type the command `mkdir` followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command `cd` followed by the folder name
- To create a new file to write the c code using editor type command `nano` followed by `.c` extension
- A Blank Document opens up into which the following source code can
- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file
- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ mkdir subtraction
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ cd subtraction/
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/subtraction$ nano sub.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/subtraction$ riscv64-unknown-elf-gcc sub.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/subtraction$ ls
a.out  sub.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/subtraction$ spike pk a.out
bbl loader
Enter two integers: 5 3
5 - 3 = 2copi-001@copi001-OptiPlex-7050:~/spike/Embedded/subtraction$
```

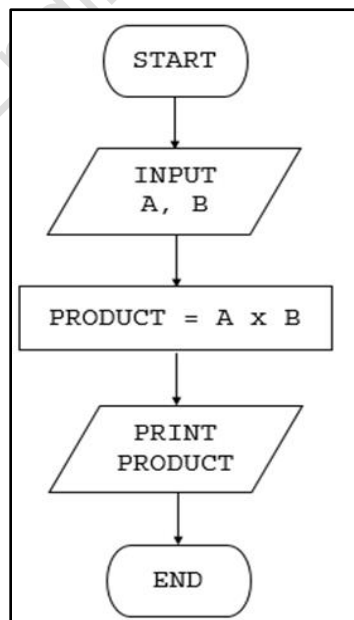
1c : C program to implement multiplication operations using RISC-V Simulator

AIM: To write a C program for subtraction and run simulation using RISC-V Simulator.

Tool Required :

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Flow chart for Multiplication



Creating a Work space :

- To create a new folder in the directory type the command mkdir followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command cd followed by the folder name
- To create a new file to write the c code using editor type command nano followed by .c extension
- A Blank Document opens up into which the following source code can be edited.

Code for multiplication :

```
#include <stdio.h>
int main() {

    int number1, number2, mul;
    printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2);

    // calculate the multiplication
    mul = number1 * number2;

    printf("%d * %d = %d", number1, number2, mul);
    return 0;
}
```

Functional Simulation:

- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file
- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ mkdir multiplication
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ cd multiplication/
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/multiplication$ nano mul.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/multiplication$ riscv64-unknown-elf-gcc mul.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/multiplication$ ls
a.out  mul.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/multiplication$ spike pk a.out
bbl loader
Enter two integers: 2 3
2 * 3 = 6copi-001@copi001-OptiPlex-7050:~/spike/Embedded/multiplication$
```

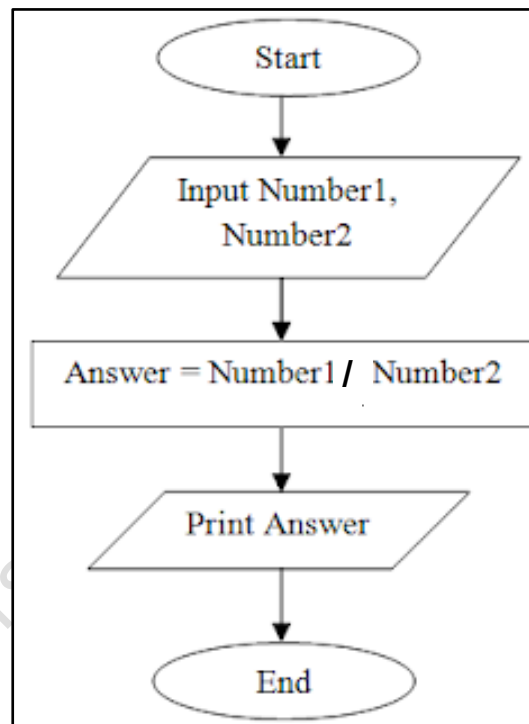
1d : C program to implement division operations using RISC-V Simulator

AIM: To write a C program for subtraction and run simulation using RISC-V Simulator.

Tool Required:

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Flow chart for Division



Creating a Work space :

- To create a new folder in the directory type the command `mkdir` followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command `cd` followed by the folder name
- To create a new file to write the c code using editor type command `nano` followed by `.c` extension
- A Blank Document opens up into which the following source code can be edited.

Code for division :

```
#include <stdio.h>
int main() {

    int number1, number2, div;
    printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2);
    // calculate the quotient
    div = number1 / number2;

    printf("%d / %d = %d", number1, number2, div);
    return 0;
}
```

- Click on Ctrl+S to save the code
- Click on Ctrl+X to exit from text editor

Functional Simulation:

- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file
- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ mkdir division
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ cd division
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/division$
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/division$ nano div.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/division$ riscv64-unknown-elf-gcc div.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/division$ ls
a.out  div.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/division$ spike pk a.out
bbl loader
Enter two integers: 4 2
4 / 2 = 2copi-001@copi001-OptiPlex-7050:~/spike/Embedded/division$
```

2: C program to implement Direct Fourier Transform using RISC-V Simulator

AIM: To write a C program for DFT and run simulation using RISC-V Simulator.

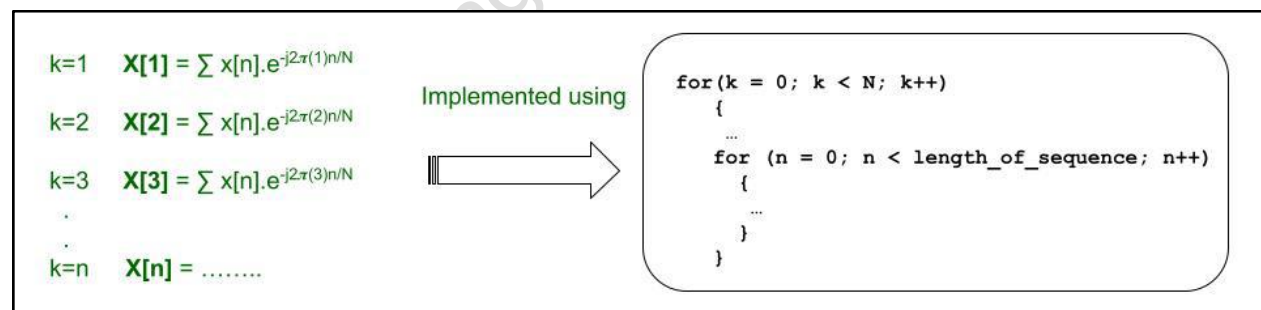
Tool Required :

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Design Information :

The discrete Fourier transform (DFT) is a basic yet very versatile algorithm for digital signal processing (DSP). The DFT overall is a function that maps a vector of n complex numbers to another vector of n complex numbers. Using 0-based indexing, let $x(t)$ denote the t th element of the input vector and let $X(k)$ denote the k th element of the output vector. Then the basic DFT is given by the following formula:

$$X(k) = \sum_{t=0}^{n-1} x(t) e^{-2\pi i t k / n}$$



Creating a Work space :

- To create a new folder in the directory type the command `mkdir` followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command `cd` followed by the folder name
- To create a new file to write the c code using editor type command `nano` followed by `.c` extension
- A Blank Document opens up into which the following source code can be edited.

Code for DFT:

```
#include <math.h>

#include <stdio.h>

void calculateDFT(int len)
{
    int xn[len];

    float Xr[len];

    float Xi[len];

    int i, k, n, N = 0;

    for (i = 0; i < len; i++) {
        printf("Enter the value of x[%d]: ", i);

        scanf("%d", &xn[i]);
    }

    printf("Enter the number of points in the DFT: ");

    scanf("%d", &N);

    for (k = 0; k < N; k++) {
        Xr[k] = 0;

        Xi[k] = 0;

        for (n = 0; n < len; n++) {
            Xr[k] = (Xr[k] + xn[n] * cos(2 * 3.141592 * k * n / N));

            Xi[k] = (Xi[k] - xn[n] * sin(2 * 3.141592 * k * n / N));
        }

        printf("(%.f) + j(%.f)\n", Xr[k], Xi[k]);
    }
}

// Driver Code
```

```
int main()
{
    int len = 0;

    printf("Enter the length of "
        "the sequence: ");

    scanf("%d4", &len);

    calculateDFT(len);

    return 0;
}
```

- Click on Ctrl+S to save the code
- Click on Ctrl+X to exit from text editor

Functional Simulation:

- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file
- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ mkdir dft
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ cd dft
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/dft$ nano dft.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/dft$ riscv64-unknown-elf-gcc dft.c -lm
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/dft$ ls
a.out  dft.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/dft$ spike pk a.out
bbl loader
Enter the length of the sequence: 4
Enter the value of x[0]: 1
Enter the value of x[1]: 4
Enter the value of x[2]: 9
Enter the value of x[3]: 16
Enter the number of points in the DFT: 4
(30.000000) + j(0.000000)
(-8.000014) + j(11.999994)
(-10.000000) + j(-0.000022)
(-7.999957) + j(-12.000017)
```

Code with resource estimation

```
#include <math.h>

#include <stdio.h>

unsigned long read_cycles(void)
{
    unsigned long cycles;

    asm volatile ("rdcycle %0" : "=r" (cycles));

    return cycles;
}

void calculateDFT(int len)
{
    int xn[len];

    float Xr[len];

    float Xi[len];

    int i, k, n, N = 0;

    for (i = 0; i < len; i++) {

        printf("Enter the value of x[%d]: ", i);

        scanf("%d", &xn[i]);

    }

    printf("Enter the number of points in the DFT: ");

    scanf("%d", &N);

    for (k = 0; k < N; k++) {

        Xr[k] = 0;

        Xi[k] = 0;

        for (n = 0; n < len; n++) {
```

```
Xr[k] = (Xr[k] + xn[n] * cos(2 * 3.141592 * k * n / N));  
Xi[k] = (Xi[k] - xn[n] * sin(2 * 3.141592 * k * n / N));  
  
}  
  
printf("(%f) + j(%f)\n", Xr[k], Xi[k]);  
  
} }
```

// Driver Code

```
int main()  
{  
  
    int len = 0;  
  
    unsigned long start, stop;  
  
    printf("Enter the length of the sequence: ");  
  
    scanf("%d4", &len);  
  
    start = read_cycles;  
  
    calculateDFT(len);  
  
    stop = read_cycles();  
  
    printf("cycle :%d\n", stop - start);  
  
    return 0;  
  
}
```

3 : C program to implement Fast Fourier Transform using RISC-V Simulator

AIM: To write a C program for FFT and run simulation using RISC-V Simulator.

Tool Required :

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Design Information :

The Fast Fourier Transform (FFT) is a mathematical technique for transforming a time-domain digital signal into a frequency-domain representation of the relative amplitude of different frequency regions in the signal. The FFT is a method for doing this process very efficiently. It may be computed using a relatively short excerpt from a signal.

The FFT is one of the most important topics in Digital Signal Processing. It is extremely important in the area of frequency(spectrum) analysis; for example, voice recognition, digital coding of acoustic signals for data stream reduction in the case of digital transmission, detection of machine vibration, signal filtration, solving partial differential equations.

This process of splitting the time-domain sequence into even and odd samples is what gives the algorithm

its name, “Decimation In Time (DIT)”. Thus, $a(m)$ and $b(m)$ are obtained by decimating $x(n)$ by a factor of two; hence, the resulting FFT algorithm is also called “radix-2”.

It is the simplest and most common form of the Cooley-Tukey algorithm

$$X(k) = \sum_{m=0}^{N/2-1} a(m) \cdot W_{N/2}^{mk} + W_N^k \sum_{m=0}^{N/2-1} b(m) \cdot W_{N/2}^{mk} = A(k) + W_N^k B(k)$$

$$0 \leq k < N$$

Creating a Work space :

- To create a new folder in the directory type the command `mkdir` followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command `cd` followed by the folder name
- To create a new file to write the c code using editor type command `nano` followed by .c extension
- A Blank Document opens up into which the following source code can be edited.

Code for FFT:

```
#include <stdio.h>

#include <math.h>

#include <complex.h>

void fft(double complex *x, int N)           // Function to perform FFT
{
    if (N <= 1)
        return;

    double complex even[N/2], odd[N/2];b    // Divide the sequence into even and odd parts
    for (int i = 0; i < N/2; i++) {
        even[i] = x[2*i];
        odd[i] = x[2*i + 1];
    }

    fft(even, N/2);                          // Recursively compute FFT for even and odd parts
    fft(odd, N/2);

    for (int k = 0; k < N/2; k++) {          // Combine the results of even and odd parts
        double complex t = cexp(-2.0 * M_PI * I * k / N) * odd[k];
        x[k] = even[k] + t;
        x[k + N/2] = even[k] - t;
    }
}

int main()
{

```

```
double complex x[] = {1, 2, 3, 4, 5, 6, 7, 8};

int N = sizeof(x) / sizeof(x[0]);           // Compute the FFT

fft(x, N);

printf("FFT Result:\n");                    // Print the FFT result

for (int i = 0; i < N; i++) {

    printf("X[%d] = %.2f + %.2fi\n", i, creal(x[i]), cimag(x[i]));

}

return 0;

}
```

- Click on Ctrl+S to save the code
- Click on Ctrl+X to exit from text editor

Functional Simulation:

- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file
- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ mkdir fft
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ cd fft
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/fft$ nano fft.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/fft$ riscv64-unknown-elf-gcc fft.c -lm
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/fft$ ls
a.out  fft.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/fft$ spike pk a.out
bbl loader
FFT Result:
X[0] = 36.00 + 0.00i
X[1] = -4.00 + 9.66i
X[2] = -4.00 + 4.00i
X[3] = -4.00 + 1.66i
X[4] = -4.00 + 0.00i
X[5] = -4.00 + -1.66i
X[6] = -4.00 + -4.00i
X[7] = -4.00 + -9.66i
```

4 : C program to implement Convolution using RISC-V Simulator

AIM: To write a C program for Convolution and run simulation using RISC-V Simulator.

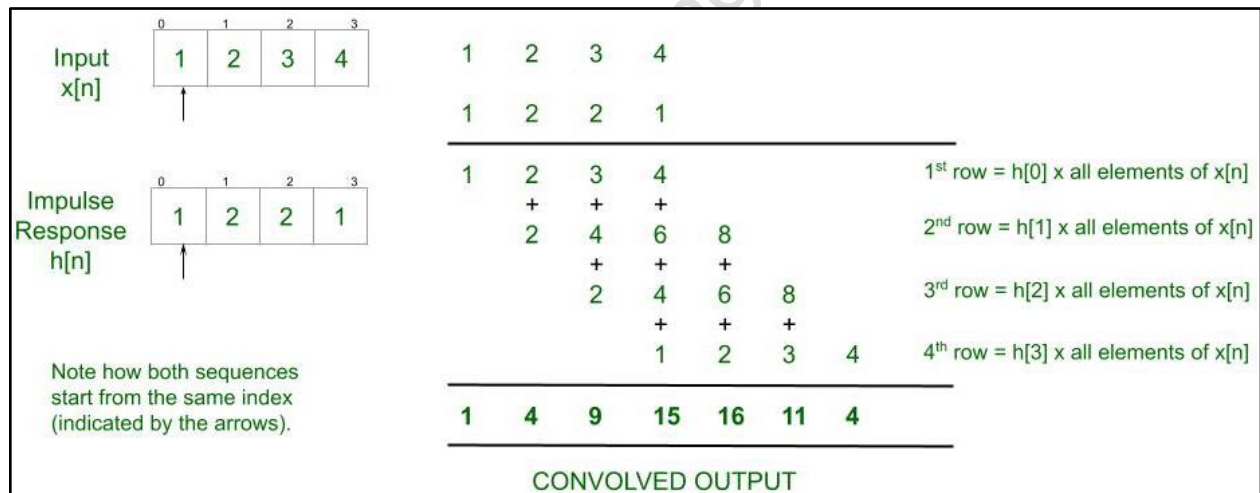
Tool Required :

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Design Information :

Convolution is the process by which one may compute the overlap of two graphs. In fact, convolution is also interpreted as the area shared by the two graphs over time. Metaphorically, it is a blend between the two functions as one passes over the other.

$$f(n) * h(n) = \sum_{k=-\infty}^{\infty} f(k) \cdot h(n - k)$$



Creating a Work space :

- To create a new folder in the directory type the command `mkdir` followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command `cd` followed by the folder name
- To create a new file to write the c code using editor type command `nano` followed by `.c` extension
- A Blank Document opens up into which the following source code can be edited.

Code for Convolution :

```
#include <math.h>
#include <stdio.h>

void calc_conv(int*, int*);
int x[10], h[10], y[10];    // Choose any length. They must all be equal though.
int l1, l2 = 0;

void main()                  // Driver code
{
    printf("Enter the length of the first sequence: ");
    scanf("%d", &l1);
    printf("Enter the length of the second sequence: ");
    scanf("%d", &l2);
    calc_conv(&l1, &l2);    // Delegating calculation to a separate function.
}

void calc_conv(int* len1, int* len2)
{
    int l = (*len1) + (*len2) - 1;
    int i, j, n, k = 0;

    for (i = 0; i < *len1; i++) {    // Getting values of 1st sequence
        printf("Enter x[%d]:", i);
        scanf("%d", &x[i]);
    }

    for (j = 0; j < *len2; j++) {    // Getting values of 2nd sequence
        printf("Enter h[%d]:", j);
        scanf("%d", &h[j]);
    }

    for (n = 0; n < l; n++) {
        y[n] = 0;
        for (k = 0; k < len1; k++) {

            if ((n - k) >= 0                // To right shift the impulse
                && (n - k) <= *len2) {
```

```

        y[n] = y[n] + x[k] * h[n - k];           // Main calculation
    }
}
printf("%d\t", y[n] );
}
}

```

- Click on Ctrl+S to save the code
- Click on Ctrl+X to exit from text editor

Functional Simulation:

- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file
- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```

copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ mkdir convolution
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ cd convolution/
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/convolution$ nano convolution.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/convolution$ riscv64-unknown-elf-gcc convolution.c -lm
convolution.c: In function 'calc_conv':
convolution.c:47:31: warning: comparison between pointer and integer
   47 |         for (k = 0; k < len1; k++) {
      |                             ^
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/convolution$ ls
a.out  convolution.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/convolution$ spike pk a.out
bbl loader
Enter the length of the first sequence: 4
Enter the length of the second sequence: 4
Enter x[0]:1
Enter x[1]:2
Enter x[2]:3
Enter x[3]:4
Enter h[0]:1
Enter h[1]:2
Enter h[2]:2
Enter h[3]:1
1      4      9      15      16      11      4      copi-001@copi001-OptiPlex-7050:~/spike/Embedded

```

5 : C program to implement bubble sort using RISC-V Simulator

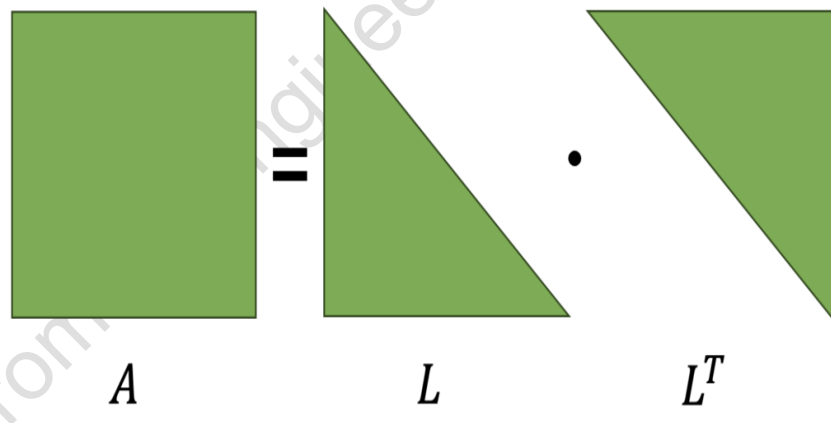
AIM: To write a C program for Cholesky Decomposition and run simulation using RISC-V Simulator.

Tool Required:

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Design Information:

Cholesky Decomposition is a numerical method used to decompose a **symmetric positive-definite matrix** into the product of a lower triangular matrix and its transpose. It is represented as $A = LL^T$, where A is the given matrix and L is the lower triangular matrix. The method is computationally efficient and is commonly used for solving systems of linear equations, numerical simulations, optimization, and covariance calculations.



Creating a Work space :

- To create a new folder in the directory type the command `mkdir` followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command `cd` followed by the folder name
- To create a new file to write the c code using editor type command `nano` followed by `.c` extension
- A Blank Document opens up into which the following source code can be edited.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
int cholesky(const double *A, double *L, int n)
{
    int i, j, k;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            L[i * n + j] = 0.0;
        }
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j <= i; j++) {
            double sum = 0.0;
            for (k = 0; k < j; k++) {
                sum += L[i * n + k] * L[j * n + k];
            }
            if (i == j) {
                double diag = A[i * n + i] - sum;
                if (diag <= 0.0) {
                    return -1;
                }
                L[i * n + j] = sqrt(diag);
            }
            else {
                L[i * n + j] =
                    (A[i * n + j] - sum) /
                    L[j * n + j];
            }
        }
    }
    return 0;
}

void print_matrix(const double *M, int n)
{
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("%10.5f", M[i * n + j]);
        }
        printf("\n");
    }
}
```

```
int main(void)
{
    double A[] = {
        4.0, 2.0, 2.0,
        2.0, 5.0, 1.0,
        2.0, 1.0, 3.0
    };
    double L[3 * 3];
    int n = 3;
    if (cholesky(A, L, n) != 0) {
        printf("Cholesky decomposition failed.\n");
        printf("Matrix is not positive definite.\n");
        return 1;
    }
    printf("Matrix A:\n");
    print_matrix(A, n);
    printf("\nCholesky factor L:\n");
    print_matrix(L, n);
    return 0;
}
```

6 : C program to implement bubble sort using RISC-V Simulator

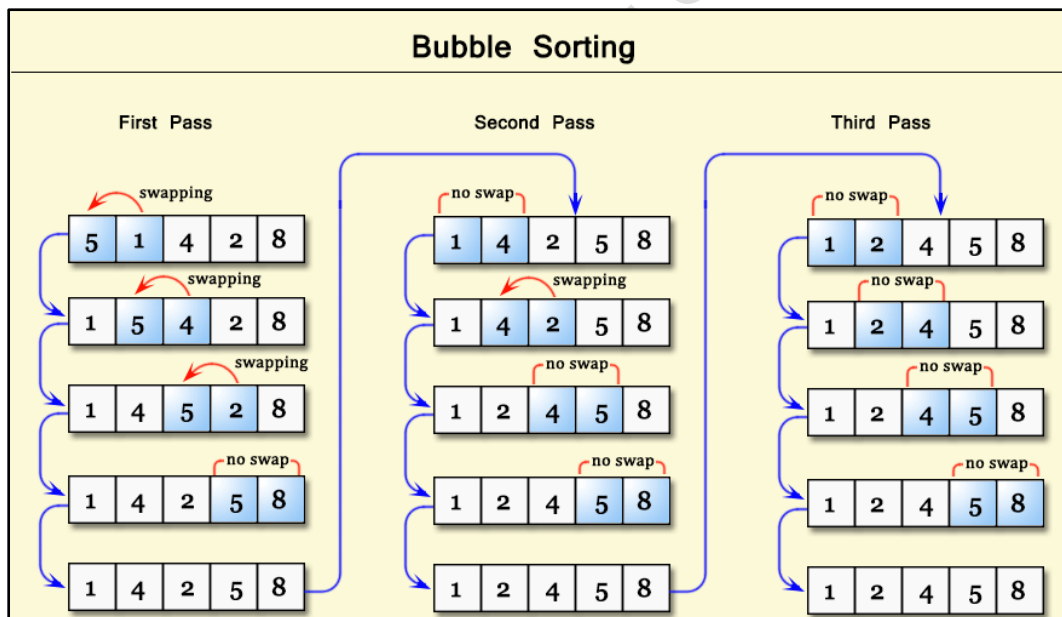
AIM: To write a C program for bubble sort and run simulation using RISC-V Simulator.

Tool Required :

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Design Information :

Bubble Sort is a comparison based simple sorting algorithm that works by comparing the adjacent elements and swapping them if the elements are not in the correct order. It is an in-place and stable sorting algorithm that can sort items in data structures such as arrays and linked lists.



Creating a Work space :

- To create a new folder in the directory type the command `mkdir` followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command `cd` followed by the folder name
- To create a new file to write the c code using editor type command `nano` followed by `.c` extension

- A Blank Document opens up into which the following source code can be edited.

Code for Bubble sort :

```
#include <stdio.h>
void swap(int* arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1])
                swap(arr, j, j + 1);
        }
    }
}
int main() {
    int arr[] = { 6, 0, 3, 5 };
    int n = sizeof(arr) / sizeof(arr[0]);

    bubbleSort(arr, n);
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

// Last i elements are already in place, so the loop
// will only run n - i - 1 times

// Calling bubble sort on array

Functional Simulation:

- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file
- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ mkdir sort
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ cd sort
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/sort$ nano sort.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/sort$ riscv64-unknown-elf-gcc sort.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/sort$ ls
a.out  sort.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/sort$ spike pk a.out
bbl loader
0 3 5 6 copi-001@copi001-OptiPlex-7050:~/spike/Embedded/sort$
```

7 : C program to implement binary search using RISC-V Simulator

AIM: To write a C program for bubble sort and run simulation using RISC-V Simulator.

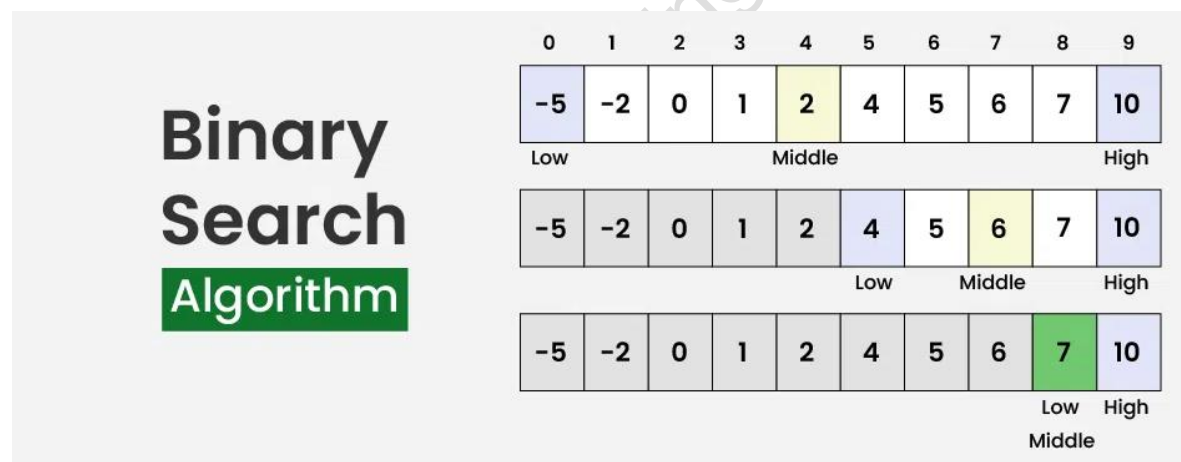
Tool Required :

- RISC-V GNU Toolchain
- RISC-V SPIKE Simulator
- RISC-V Proxy Kernel

Design Information :

Binary search is a search algorithm used to find the position of a target value within a **sorted** array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty.

The search interval is halved by comparing the target element with the middle value of the search space.



Creating a Work space :

- To create a new folder in the directory type the command `mkdir` followed by the folder name, in which the saved program will be stored
- To access the newly created folder type the command `cd` followed by the folder name
- To create a new file to write the c code using editor type command `nano` followed by `.c` extension
- A Blank Document opens up into which the following source code can be edited.

Code for Binary search :

```
#include <stdio.h>

int binarySearch(int array[], int x, int low, int high) {
    while (low <= high) {          // Repeat until the pointers low and high meet each other
        int mid = low + (high - low) / 2;
        if (array[mid] == x)
            return mid;
        if (array[mid] < x)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main(void) {
    int array[] = {3, 4, 5, 6, 7, 8, 9};
    int n = sizeof(array) / sizeof(array[0]);
    int x = 4;
    int result = binarySearch(array, x, 0, n - 1);
    if (result == -1)
        printf("Not found");
    else
        printf("Element is found at index %d", result);
    return 0;
}
```

Functional Simulation:

- To invoke the RISC-V Toolchain to compile the C program and generate Executable Linked Format (ELF) file type the following command followed the saved c program file
- The generated ELF file is given as input to SPIKE simulator and run the simulation using the following command

```
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ mkdir binary
copi-001@copi001-OptiPlex-7050:~/spike/Embedded$ cd binary/
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/binary$ nano search.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/binary$ riscv64-unknown-elf-gcc search.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/binary$ ls
a.out  search.c
copi-001@copi001-OptiPlex-7050:~/spike/Embedded/binary$ spike pk a.out
bbl loader
Element is found at index 1copi-001@copi001-OptiPlex-7050:~/spike/Embedded/binary$
```